Real-Time Speech Translation

Cornerstone Project

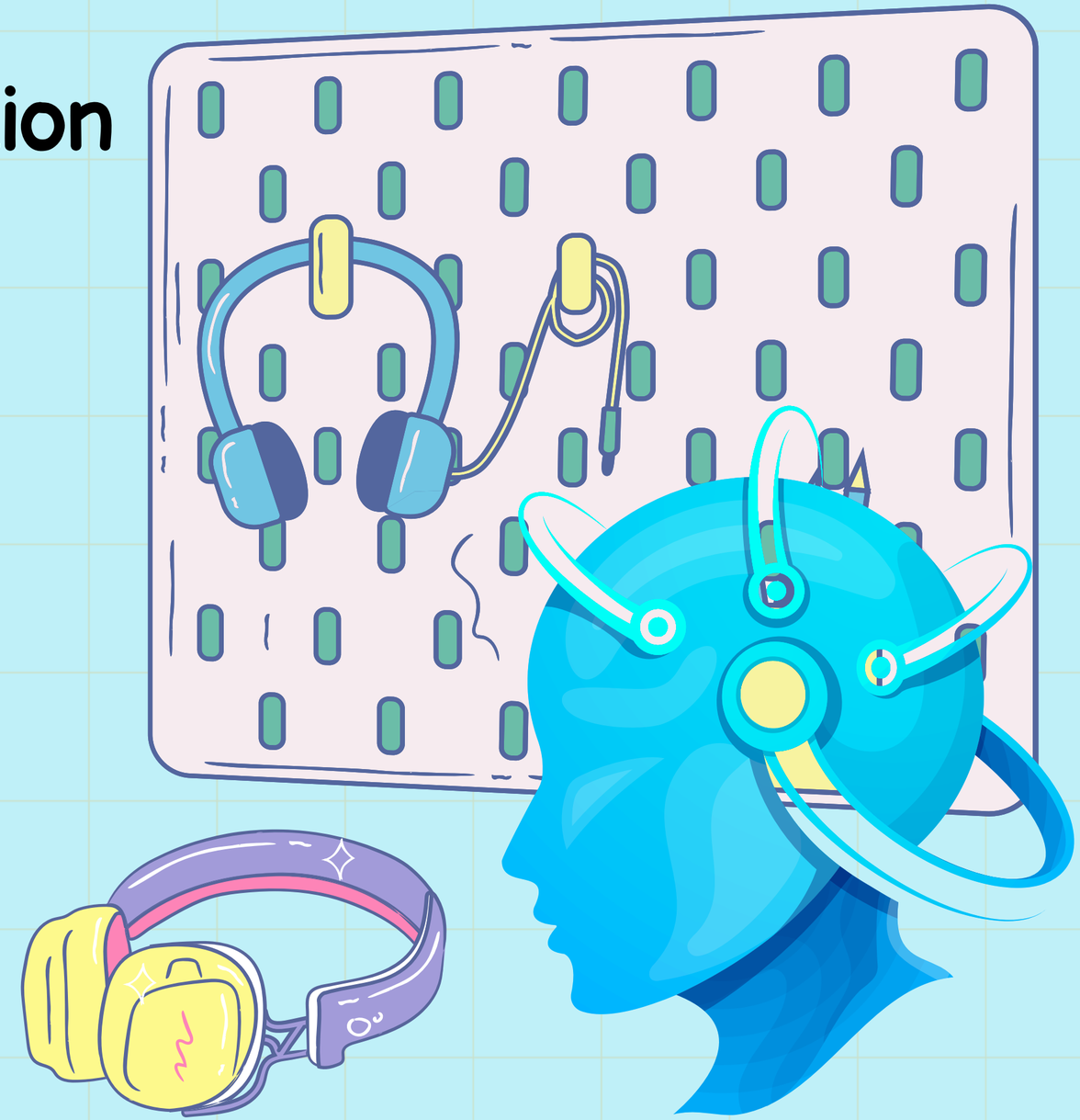
INSTRUCTOR

PROF KOLIN PAUL

GROUP

ASTITWA SAXENA – 2025MCS3005

ABHISHEK GUPTA – 2025MCS2963

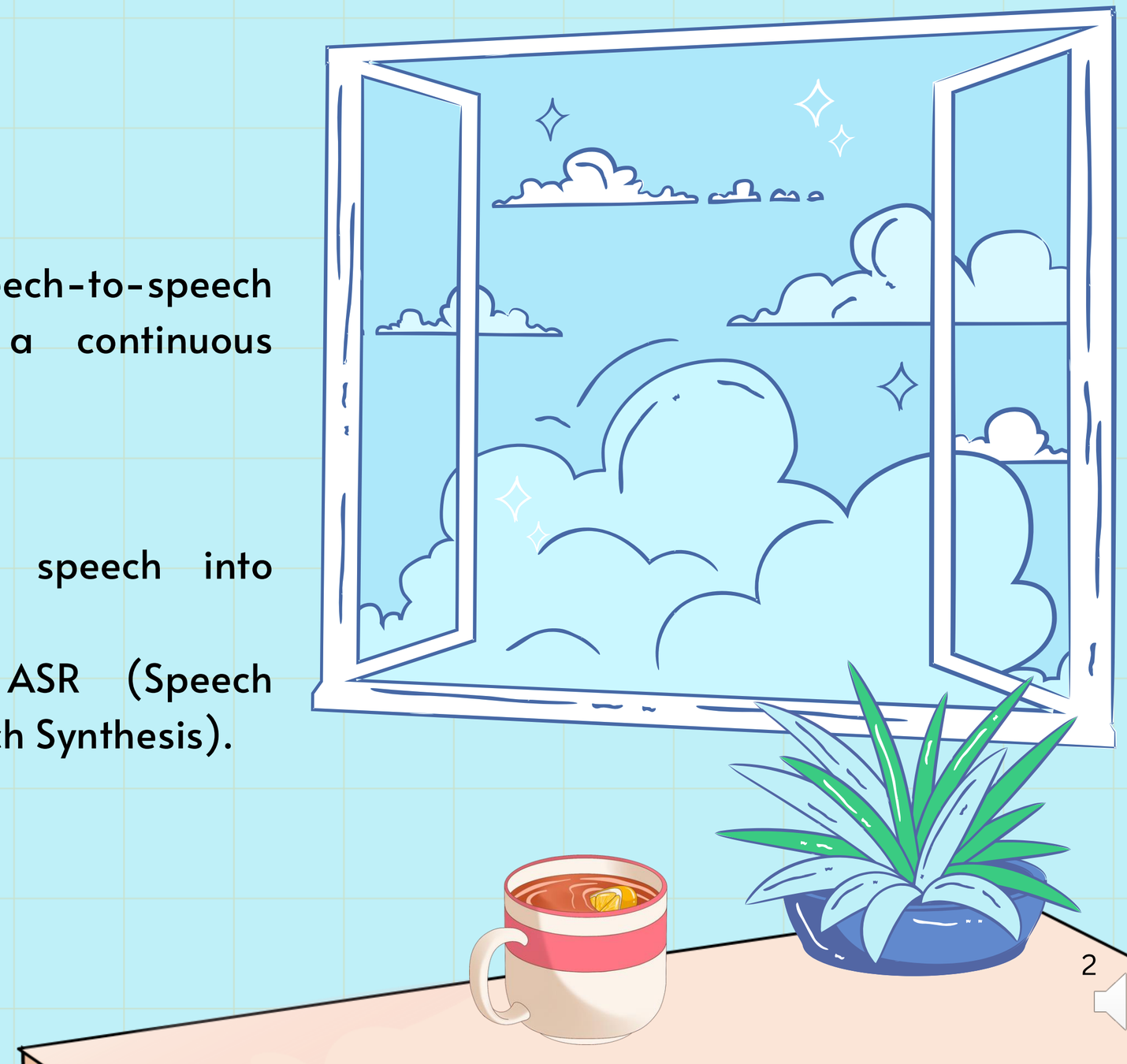


Objective

The project aims to build a real-time speech-to-speech translation system that operates as a continuous processing pipeline.

Key Features:

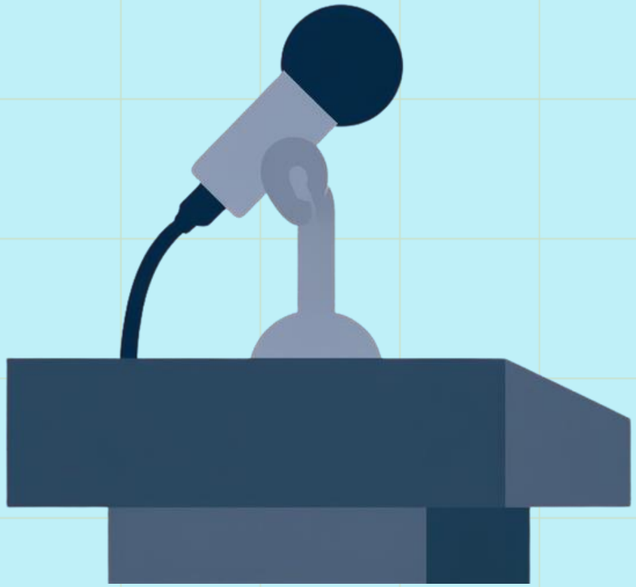
- Real-time processing: Converts live speech into translated speech with minimal latency.
- Modules involved: Microphone → ASR (Speech Recognition) → Translation → TTS (Speech Synthesis).



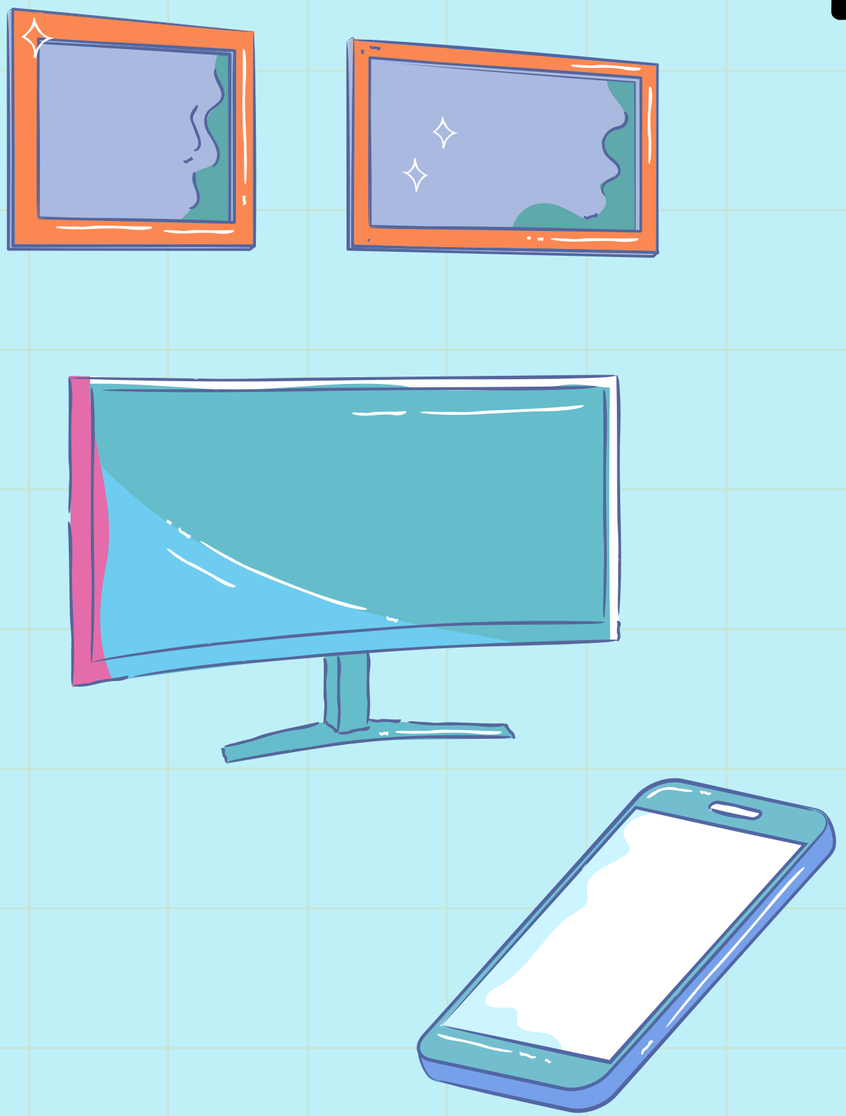
Problem Statement

Real-time speech translation is challenging due to:

- Latency constraints
- Continuous audio streaming
- Synchronization across modules



Design and Architecture



- Each module in the system runs as an independent stage.
- The core pipeline manages the execution and communication between stages via queues.

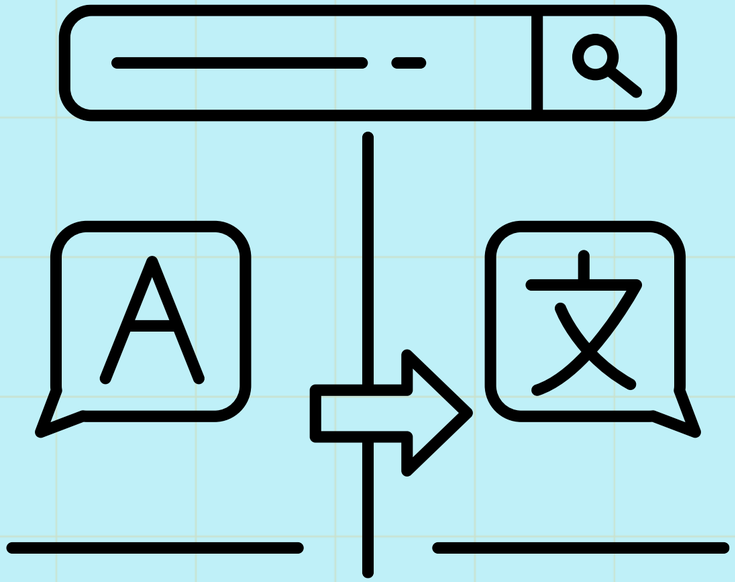
Modules:

- Microphone (input capture)
- ASR (speech-to-text)
- Translation (language conversion)
- TTS (text-to-speech output)

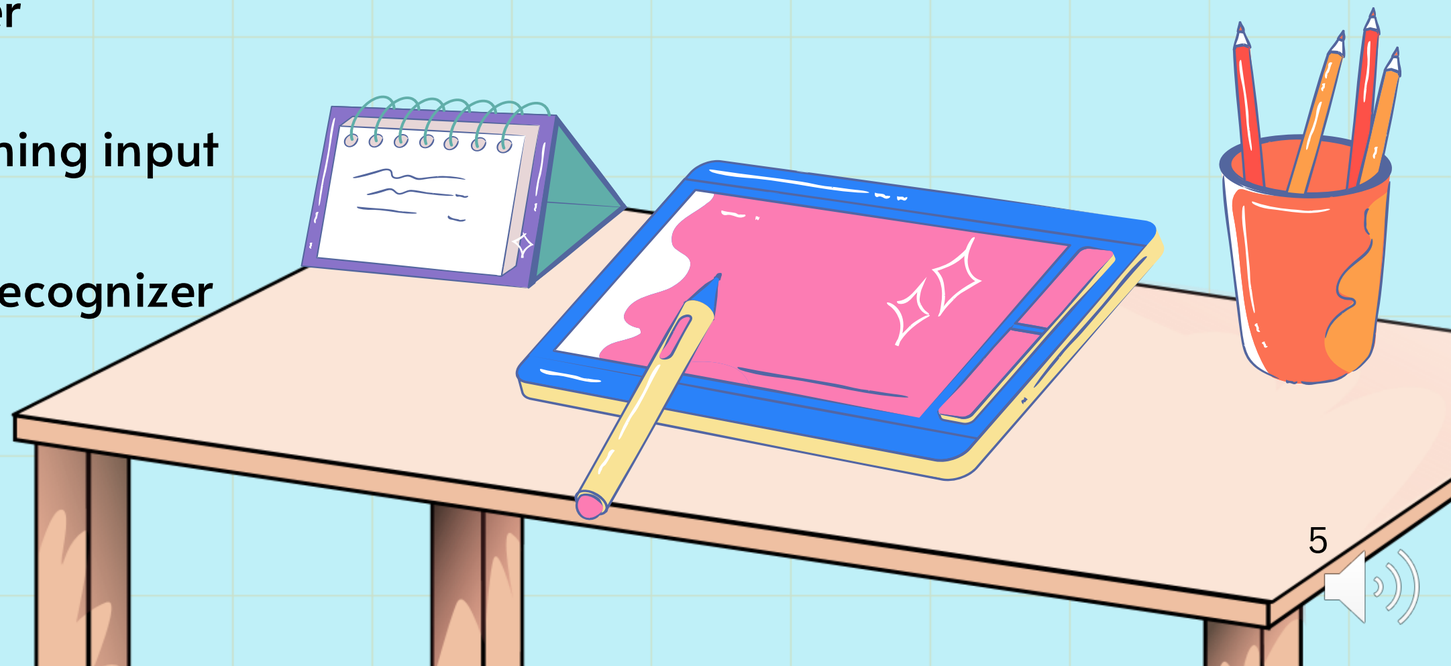
Benefits of Architecture:

- Non-blocking execution ensures minimal latency.
- Loose coupling allows modules to be easily swapped or upgraded.

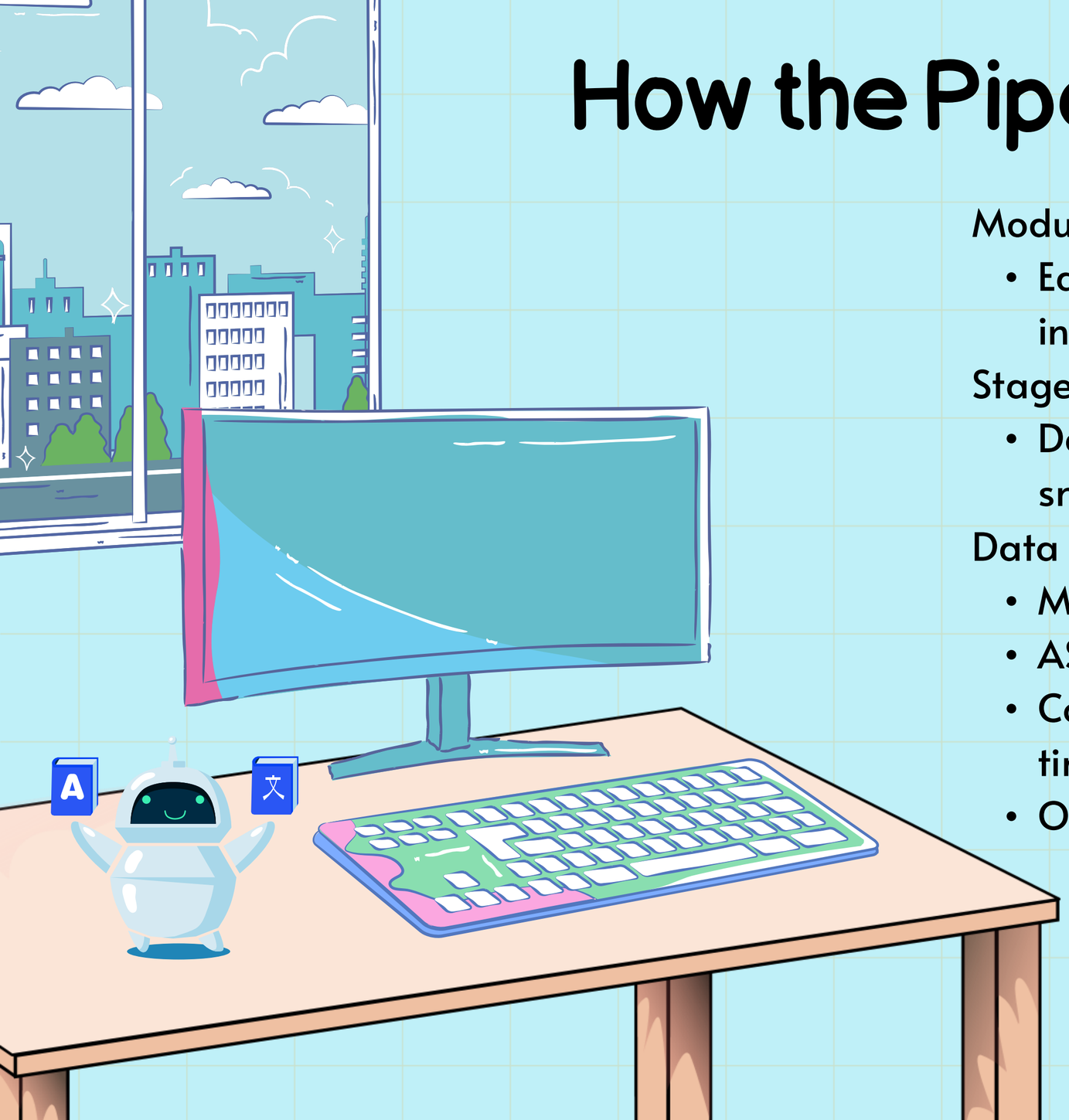
Repository Structure




```
speech-pipeline/
├── core/
│   ├── stage.py       # Base stage class for all modules
│   ├── pipeline.py    # Pipeline controller
├── audio/
│   └── mic.py         # Microphone streaming input
├── asr/
│   └── dummy_asr.py   # Placeholder recognizer
└── main.py            # Entry point
```



How the Pipeline Works



Modular Pipeline:

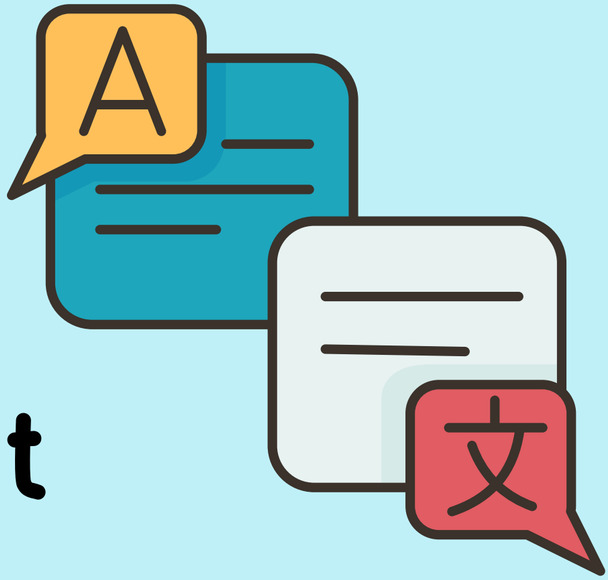
- Each stage in the pipeline follows the same contract: input → process → output

Stages and Execution:

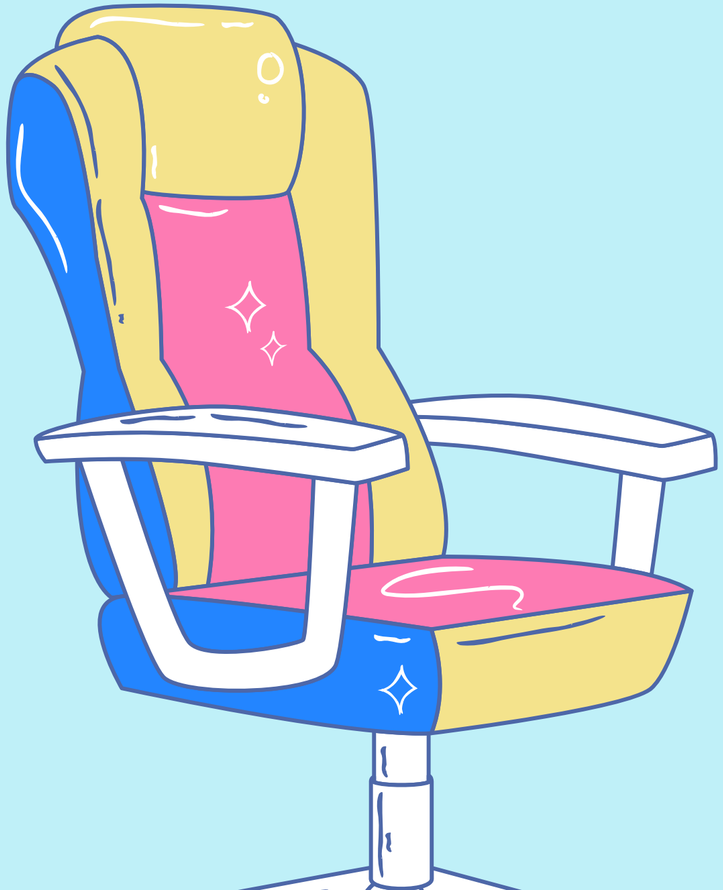
- Data is passed between stages via queues, ensuring smooth and non-blocking execution.

Data Flow Example :

- Microphone Capture: Captures live speech.
- ASR Stage: Converts speech to text
- Console Output: Displays recognized text in real-time.
- Output speech: real time converted text.



Running the Project



Install Dependencies:

- pip install sounddevice vosk

Run the Project:

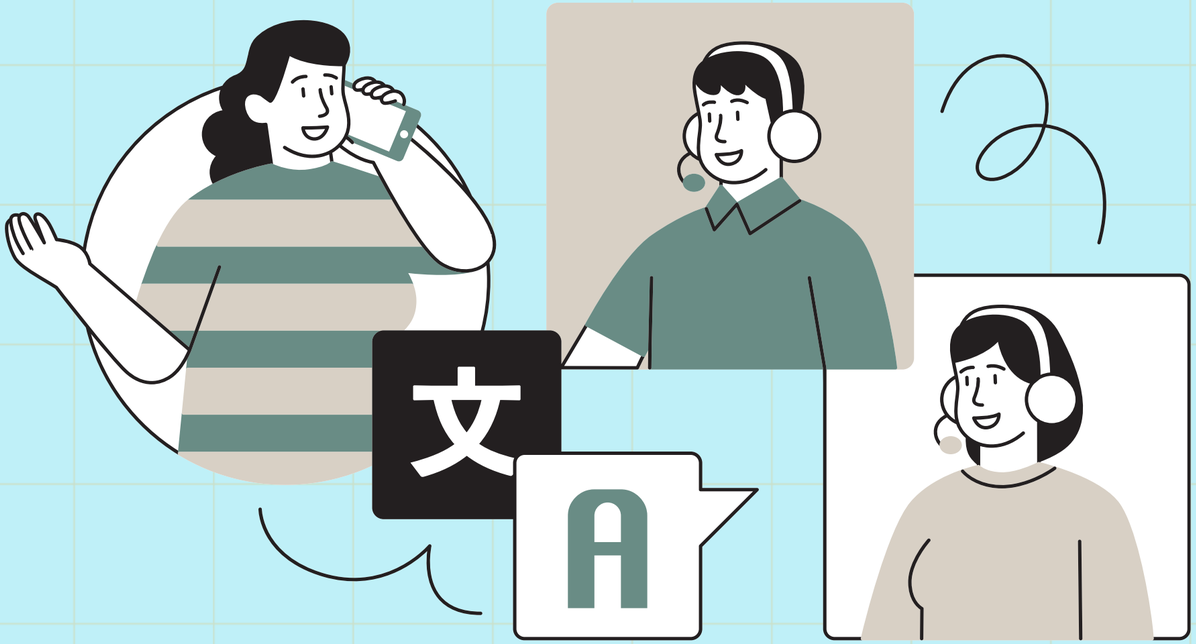
- python main.py

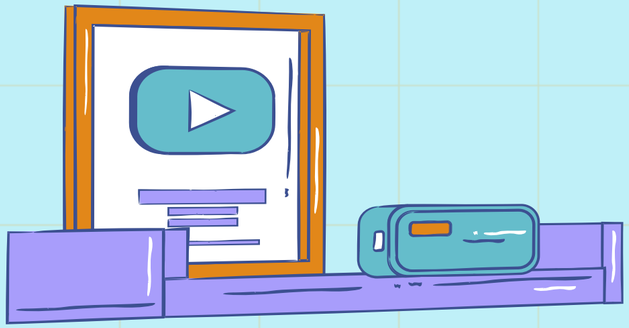
Expected Output:

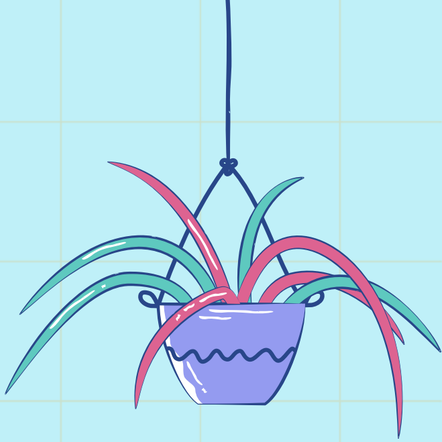
- Running pipeline...
- TEXT: recognized speech chunk
- TEXT: recognized speech chunk

Phase I — Infrastructure (Completed)

- Pipeline Engine: Core infrastructure for concurrent execution.
- Streaming Queues: Communication between stages via queues.
- Microphone Capture: Successfully capturing live audio input.
- Dummy ASR Integration: Placeholder ASR to validate system flow.





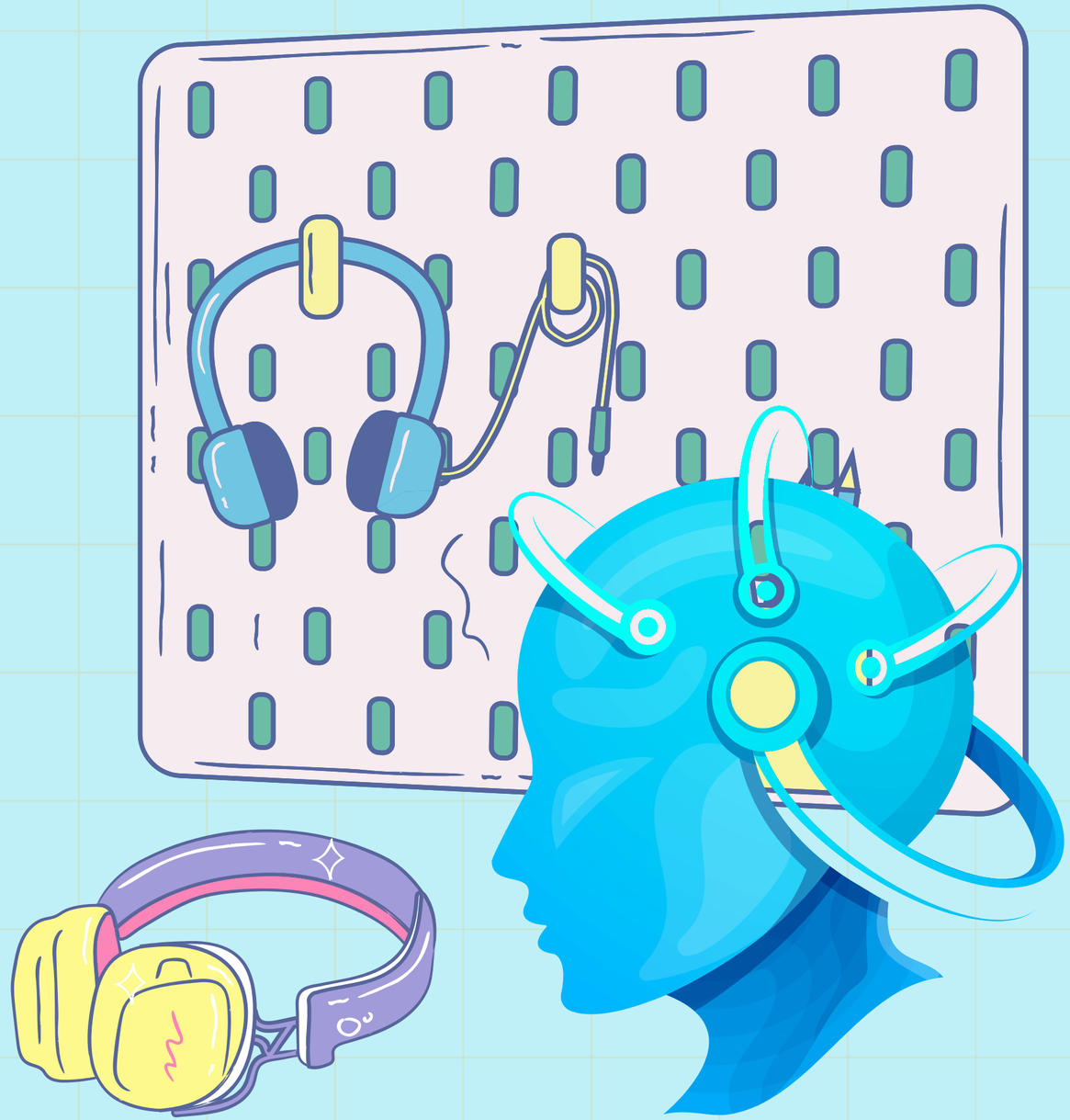


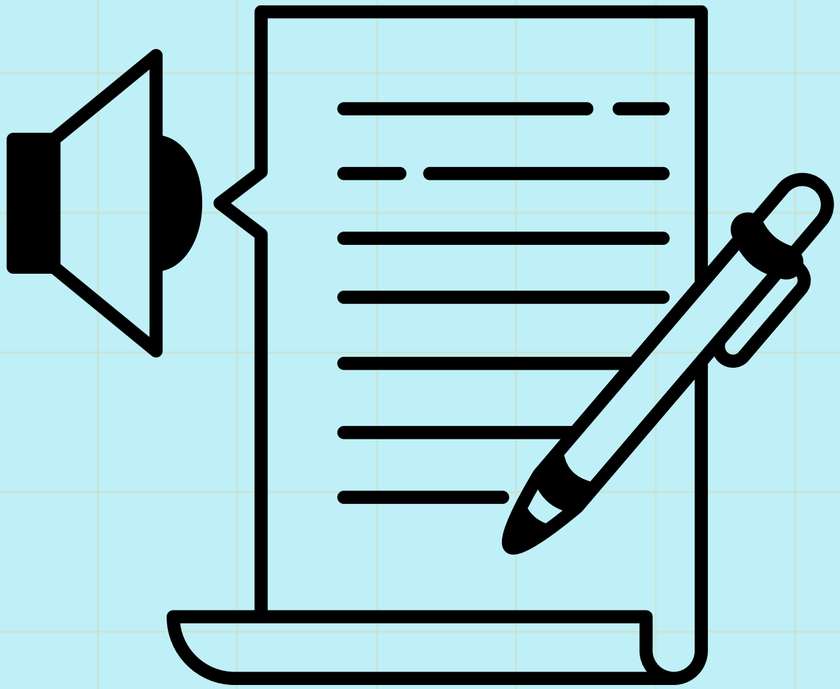
Phase 2: Real Speech Recognition

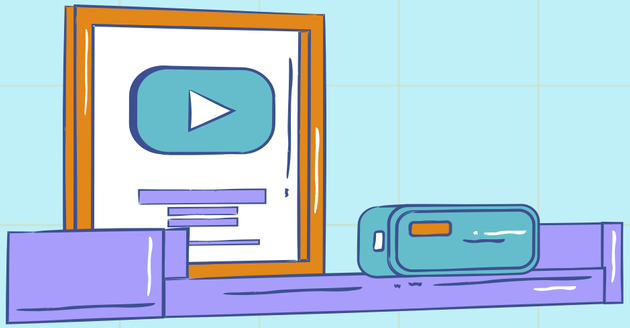
- Replaced the dummy ASR with a real ASR solution (Vosk).
- Ensure real-time speech recognition.

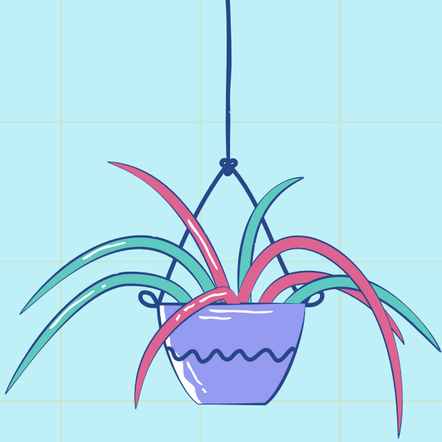
Phase 3: Translation Stage

- Added a translation module to convert recognized speech into a different language.







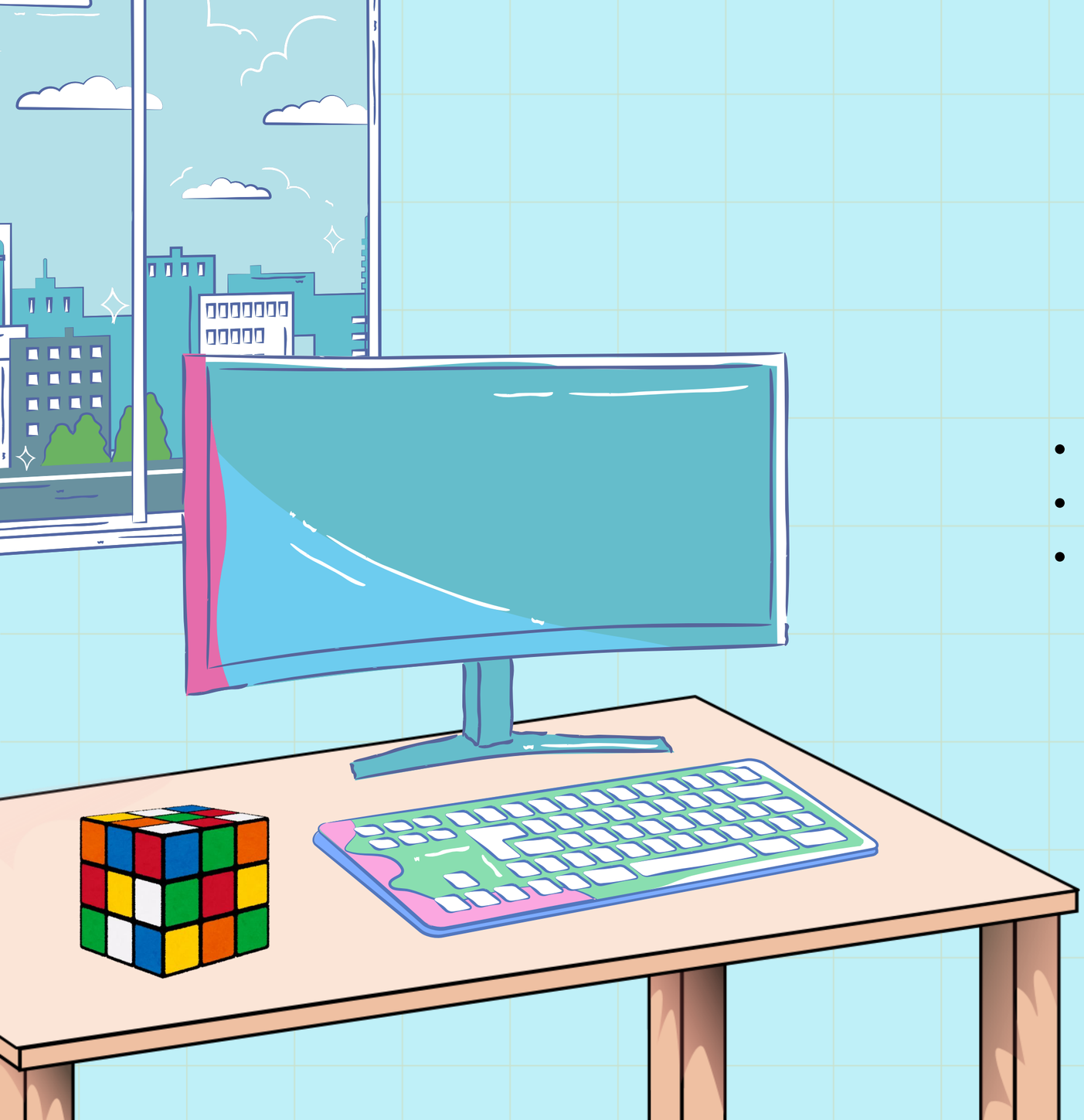


Phase 4: Speech Synthesis (in progress)

- Implemented TTS (Text-to-Speech) for converting translated text into speech.
- Now Focusing on buffering, and smooth streaming.

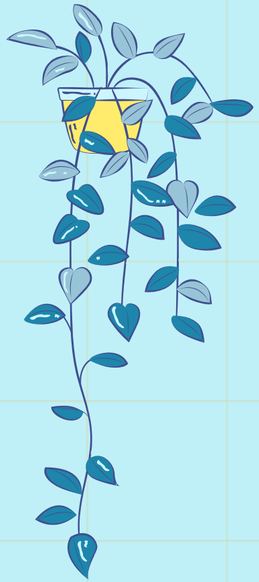
Phase 5: Dashboard

Add live system monitor



Key Challenges

- Challenge 1: Real-time Processing Latency
- Challenge 2: Accuracy of ASR
- Challenge 3: Managing Synchronization Between Stages

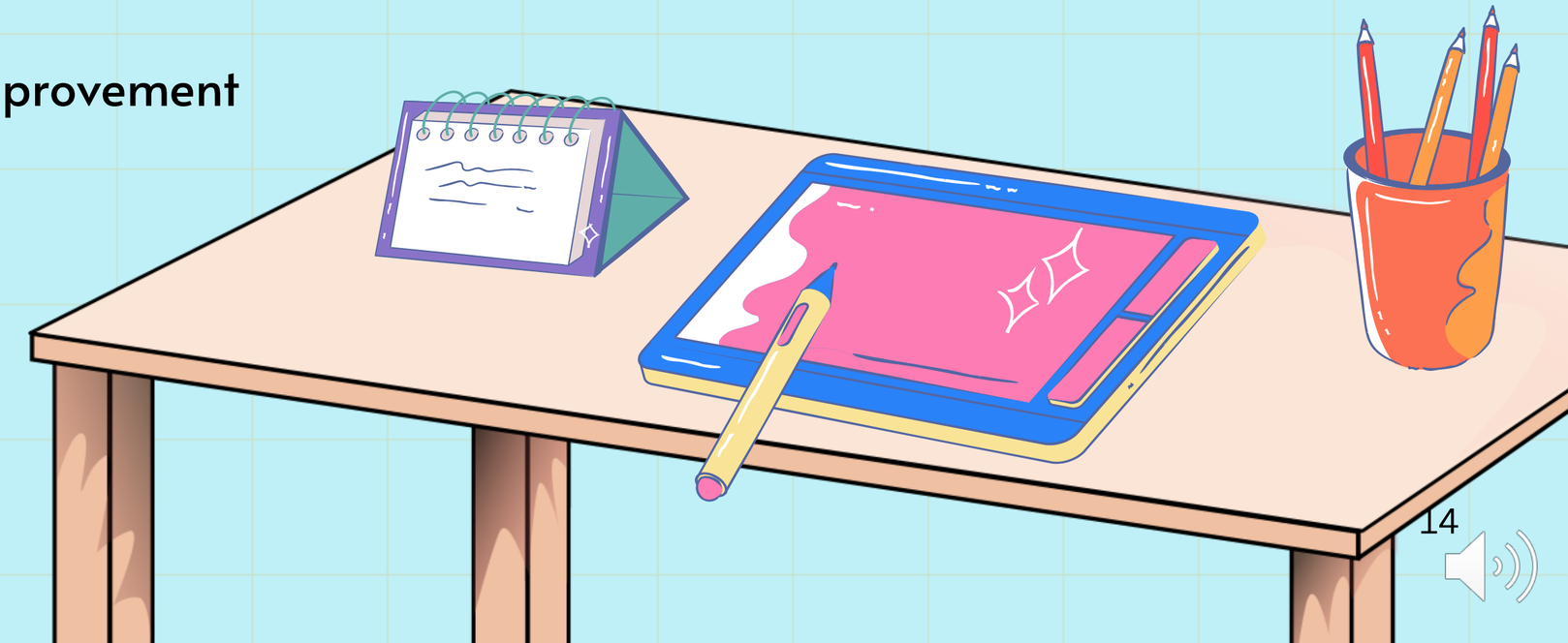


Conclusion

- We've established the core infrastructure for the real-time speech-to-speech translation system.
- The current system works with live microphone input.

Next Milestones:

- Translation improvement.
- TTS for real-time speech improvement
- GUI integration



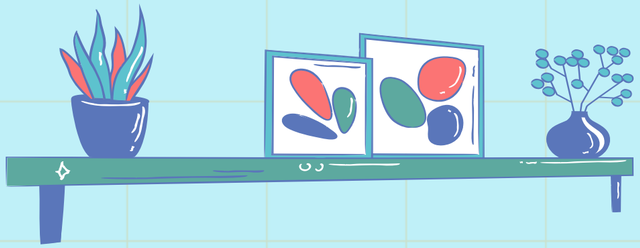
Microsoft Teams

Recording

2026-03-21 07:06 UTC

Recorded by
Astitwa Saxena

Organized by
Astitwa Saxena



Thank You